Manuel de référence

Le format JSON

JavaScript Object Notation

Normes, validation, pièges de sécurité,
JavaScript/TypeScript, Python et PostgreSQL

Node.js 22 · TypeScript · Python 3.10+ · PostgreSQL · Exemples testés

Teufack Doury Cantor
Master en data science et machine learning

Version 1.0 · Octobre 2026

Table des matières

1 À propos de ce manuel **3**
1.1 Public visé . . . 3
1.2 Comment le lire . . . 3
1.3 Les exemples . . . 3

2 Le format JSON et ses normes **4**
2.1 Origines . . . 4
2.2 Les textes de référence . . . 4
2.3 JSON n'est pas du JavaScript . . . 4

3 Grammaire et types **5**
3.1 Structure d'un texte JSON . . . 5
3.2 Erreurs de syntaxe courantes . . . 6
3.3 Les six types . . . 6

4 JSON en JavaScript et TypeScript **8**
4.1 Sérialiser avec `JSON.stringify` . . . 8
4.2 Contrôler la forme avec `toJSON` . . . 8
4.3 Les erreurs de sérialisation . . . 9
4.4 Lire avec `JSON.parse` . . . 10
4.5 Gérer les erreurs de lecture . . . 11
4.6 Copier un objet : JSON ou `structuredClone` . . . 12
4.7 Typer et valider en TypeScript . . . 13

5 JSON en Python **15**
5.1 Le module `json` . . . 15
5.2 Les paramètres de `json.dumps` . . . 16
5.3 Gérer les erreurs de lecture . . . 17
5.4 Types non pris en charge . . . 17
5.5 Valider avec Pydantic . . . 19
5.6 Bibliothèques plus rapides . . . 20

6 Valider avec JSON Schema **21**
6.1 Rôle . . . 21
6.2 Un schéma complet . . . 21
6.3 Combiner et réutiliser . . . 22
6.4 Valider en Python et en JavaScript . . . 22
6.5 Générer le schéma depuis le code . . . 23
6.6 Contraindre la réponse d'un modèle de langage . . . 23

7 Formats et standards dérivés **25**
7.1 JSON Pointer (RFC 6901) . . . 25
7.2 JSON Patch (RFC 6902) et JSON Merge Patch (RFC 7396) . . . 25
7.3 JSONPath (RFC 9535) . . . 26
7.4 JSON Lines . . . 26
7.5 JSON-LD et JWT . . . 27

8 Stocker du JSON dans PostgreSQL **29**
8.1 `json` ou `jsonb` . . . 29
8.2 Requêtes essentielles . . . 29
8.3 Quand utiliser `jsonb` . . . 30

9 Gros volumes et performance **32**
9.1 Le coût d'un chargement complet . . . 32
9.2 Lire par flux . . . 32
9.3 Compression . . . 32

10 Sécurité **33**
10.1 Imbrication profonde et documents géants . . . 33
10.2 Les grands entiers . . . 33

10.3 Clés dupliquées . . . 33
10.4 Pollution de prototype en JavaScript . . . 34
10.5 Format des erreurs d'API (RFC 9457) . . . 35

11 Choisir un format **36**
11.1 Comparaison . . . 36
11.2 Recommandations . . . 36

12 Outils en ligne de commande **37**
12.1 jq . . . 37
12.2 Valider un fichier avec Python . . . 37
12.3 PowerShell . . . 37

13 Conclusion **38**

14 Fiche de référence **39**
14.1 Règles de syntaxe . . . 39

Références bibliographiques **40**

1 À propos de ce manuel

1.1 Public visé

Ce manuel s'adresse aux développeurs qui manipulent du JSON tous les jours : appels d'API, fichiers de configuration, échanges entre une interface JavaScript et un serveur Python, colonnes de base de données. Il suppose de connaître les bases de JavaScript ou de Python. Il ne cherche pas à tout dire sur JSON. Il rassemble les règles des normes, les fonctions utiles dans chaque langage et, surtout, les pièges qui produisent des bogues en production.

1.2 Comment le lire

Les sections se lisent indépendamment. Les sections 2 et 3 posent les règles du format. Les sections 4 et 5 couvrent JavaScript, TypeScript et Python. La section 6 traite de la validation, la section 8 du stockage dans PostgreSQL et la section 10 des risques de sécurité. La fiche de référence de la section 14 résume l'essentiel.

1.3 Les exemples

Chaque bloc de code dont la légende indique un chemin, comme `exemples/python/01_dumps.py`, est un fichier réel du dossier `exemples/` livré avec ce manuel. Le document inclut ces fichiers directement : le code imprimé est celui qui a été exécuté. Une suite de tests lance chaque exemple et vérifie sa sortie (voir `exemples/README.md`). Les sorties indiquées en commentaire ont été obtenues avec Python 3.13, Node.js 22, PostgreSQL 16, Zod 4 et Pydantic 2. Les commandes de terminal sont écrites pour PowerShell sous Windows.

2 Le format JSON et ses normes

2.1 Origines

JSON (*JavaScript Object Notation*) est apparu au début des années 2000. Douglas Crockford l'a décrit comme un sous-ensemble de la notation des objets littéraux de JavaScript, assez simple pour être lu par n'importe quel langage [7]. Il a progressivement remplacé XML dans les API web, parce qu'un document JSON se convertit directement en dictionnaires, en tableaux et en valeurs simples dans la plupart des langages, sans bibliothèque lourde.

2.2 Les textes de référence

Plusieurs documents décrivent JSON (tableau 1). Ils sont compatibles entre eux, et la RFC 8259 est celui à consulter en pratique : elle contient la grammaire, les règles d'encodage et les recommandations d'interopérabilité que ce manuel cite souvent.

Document	Date	Apport
RFC 4627 [6]	juillet 2006	Premier texte, informatif. Définit le type de média `application/json` et impose un objet ou un tableau à la racine.
RFC 7159 [1]	mars 2014	Autorise n'importe quelle valeur à la racine, un nombre seul par exemple.
RFC 8259 [3]	décembre 2017	Version actuelle, classée Internet Standard (STD 90). Impose l'UTF-8 entre systèmes ouverts et interdit d'ajouter un BOM.
ECMA-404 [8]	décembre 2017	Deuxième édition. Décrit la grammaire seule, sans règles réseau.
ISO/IEC 21778 [13]	2017	Reprend ECMA-404 comme norme internationale.

Tableau 1 : Les textes qui définissent JSON.

2.3 JSON n'est pas du JavaScript

Un texte JSON ressemble à un objet JavaScript, mais ses règles sont plus strictes. Les clés doivent être entre guillemets doubles droits (`"`), alors que JavaScript accepte des clés sans guillemets ou entre apostrophes. Les chaînes n'acceptent ni apostrophes ni accents graves (*backticks*). La virgule après le dernier élément est interdite, tout comme les commentaires. Douglas Crockford a expliqué en 2012 avoir retiré les commentaires parce que certains développeurs s'en servaient pour transmettre des directives à leurs outils.

JSON ne connaît que six types (section 3.3). `undefined`, `Symbol`, `BigInt`, les fonctions, les dates et les expressions régulières n'ont pas d'équivalent. Selon le cas, `JSON.stringify` les ignore, les convertit ou lève une erreur (section 4). Enfin, un texte JSON se lit toujours avec un parseur (`JSON.parse`, `json.loads`), jamais avec `eval`, qui exécuterait le code qu'un attaquant aurait glissé dans le texte.

3 Grammaire et types

3.1 Structure d'un texte JSON

Un texte JSON contient une seule valeur, entourée d'espaces facultatifs. Seuls quatre caractères comptent comme espaces : l'espace (U+0020), la tabulation (U+0009), le saut de ligne (U+000A) et le retour chariot (U+000D). La RFC 8259 décrit la grammaire en ABNF, la notation de grammaire des RFC définie par la RFC 5234. Voici ses règles principales, simplifiées :

```
JSON-text = ws value ws
value     = false / null / true / object / array / number / string
object    = begin-object [ member *( value-separator member ) ] end-object
member    = string name-separator value
array     = begin-array [ value *( value-separator value ) ] end-array
number    = [ minus ] int [ frac ] [ exp ]
int       = zero / ( digit1-9 *DIGIT )
frac      = decimal-point 1*DIGIT
exp       = e [ minus / plus ] 1*DIGIT
string    = quotation-mark *char quotation-mark
ws        = *( %x20 / %x09 / %x0A / %x0D )
```

Code 1 : Extrait simplifié de la grammaire ABNF de la RFC 8259

La règle `int` explique deux erreurs fréquentes : un nombre ne peut pas commencer par un zéro suivi d'autres chiffres, et seul le signe moins est autorisé devant un nombre. Le code 2 montre un document qui utilise les six types.

```json
{
  "utilisateurId": 1048576,
  "pseudonyme": "AdaLovelace",
  "estActif": true,
  "profil": {
    "prenom": "Ada",
    "nom": "Byron",
    "titre": null
  },
  "roles": ["chercheuse", "pionniere", "admin"],
  "coordonnees": {
    "latitude": 51.5074,
    "longitude": -0.1278
  }
}
```

Code 2 : Un document qui utilise les six types (`exemples/json/utilisateur.json`)

3.2 Erreurs de syntaxe courantes

Le tableau 2 rassemble les erreurs qui reviennent le plus souvent, en particulier quand un texte JSON est écrit à la main ou copié depuis du code JavaScript.

Erreur	Invalide	Correct
Clé sans guillemets	`{id: 42}`	`{"id": 42}`
Apostrophes	`{'cle': 'valeur'}`	`{"cle": "valeur"}`
Virgule finale (objet)	`{"a": 1, "b": 2,}`	`{"a": 1, "b": 2}`
Virgule finale (tableau)	`[1, 2, 3,]`	`[1, 2, 3]`
Zéro en tête	`{"compteur": 007}`	`{"compteur": 7}`
Décimal sans partie entière	`{"taux": .25}`	`{"taux": 0.25}`
Valeur `undefined`	`{"statut": undefined}`	`{"statut": null}` ou clé omise
Commentaire	`// port` `{"port": 8080}`	Supprimer le commentaire

Tableau 2 : Erreurs de syntaxe fréquentes et leur correction.

3.3 Les six types

3.3.1 Chaîne (`string`)

Une chaîne est délimitée par des guillemets doubles. Tout caractère Unicode peut y figurer tel quel, sauf le guillemet, la barre oblique inversée et les caractères de contrôle U+0000 à U+001F, qui doivent être échappés. Le tableau 6 de la fiche de référence liste les séquences d'échappement. La forme `\uXXXX` désigne un caractère par son code hexadécimal. Au-delà de U+FFFF, pour les émojis par exemple, il faut une paire de substitution UTF-16 : la fusée U+1F680 s'écrit `"\uD83D\uDE80"`.

3.3.2 Nombre (`number`)

JSON ne distingue pas les entiers des décimaux. Un nombre a une partie entière, éventuellement précédée d'un signe moins, une partie décimale facultative après un point, et un exposant facultatif introduit par `e` ou `E`. Trois écritures sont interdites : un zéro en tête (`012`), le signe plus (`+5`), et les valeurs spéciales `NaN` et `Infinity` de la norme IEEE 754 [12]. La norme ne fixe ni taille ni précision. La section 10.2 montre pourquoi cela pose problème avec les grands entiers.

3.3.3 Booléen et `null`

Les booléens s'écrivent `true` et `false`, en minuscules. `null`, aussi en minuscules, indique qu'un champ existe mais n'a pas de valeur. C'est différent d'une clé absente : `{"titre": null}` dit « pas de titre », alors qu'un objet sans clé `titre` ne dit rien sur le titre.

3.3.4 Objet (`object`)

Un objet est une collection non ordonnée de paires clé-valeur entre accolades. Chaque clé est une chaîne, suivie de deux-points et d'une valeur, et les paires sont séparées par des virgules. L'ordre des clés n'a pas de signification : JavaScript et Python le conservent à la lecture, mais d'autres outils le changent, comme le type `jsonb` de PostgreSQL (section 8).

La RFC 8259 recommande que les clés d'un même objet soient uniques, sans l'imposer. Face à une clé en double, certains parseurs gardent la dernière valeur, d'autres la première, d'autres refusent le document. La section 10.3 montre comment un attaquant exploite ces différences.

3.3.5 Tableau (`array`)

Un tableau est une liste ordonnée de valeurs entre crochets, séparées par des virgules. Les valeurs peuvent être de types différents, y compris des objets et d'autres tableaux.

4 JSON en JavaScript et TypeScript

4.1 Sérialiser avec `JSON.stringify`

`JSON.stringify(valeur, replacer, space)` transforme une valeur JavaScript en texte JSON [9]. Le deuxième paramètre, `replacer`, choisit ce qui est sérialisé. Un tableau de noms sert de liste blanche : seules ces propriétés sont gardées. Une fonction `(cle, valeur) => ...` est appelée pour chaque propriété, et renvoyer `undefined` supprime la propriété. Le troisième paramètre, `space`, règle l'indentation : un nombre d'espaces entre 1 et 10, ou une chaîne dont les dix premiers caractères servent d'indentation. Sans lui, la sortie est compacte.

```
const utilisateur = {
  id: 101,
  nom: "Martin",
  motDePasseHash: "9f8379c92f3402",
  secretBancaire: "GA21-0000-0000",
  roles: ["editeur", "visiteur"],
};

// 1. Liste blanche : seules les proprietes nommees sont serialisees
console.log(JSON.stringify(utilisateur, ["id", "nom"]));
// {"id":101,"nom":"Martin"}

// 2. Fonction replacer : renvoyer undefined supprime la cle
const masque = JSON.stringify(utilisateur, (cle, valeur) => {
  const c = cle.toLowerCase();
  if (c.includes("secret") || c.includes("hash")) return undefined;
  return valeur;
});
console.log(masque);
// {"id":101,"nom":"Martin","roles":["editeur","visiteur"]}

// 3. Troisieme parametre : indentation de 2 espaces
console.log(JSON.stringify({ a: 1, b: [true, null] }, null, 2));
```

Code 3 : Liste blanche, fonction `replacer` et indentation (`exemples/js/01_replacer.mjs`)

Pour masquer des données sensibles, une liste blanche est plus sûre qu'une fonction qui retire des champs : un champ sensible ajouté plus tard au modèle ne sortira pas par accident.

4.2 Contrôler la forme avec `toJSON`

Si un objet possède une méthode `toJSON()`, `JSON.stringify` sérialise la valeur qu'elle renvoie au lieu de l'objet lui-même. `Date` utilise ce mécanisme : sa méthode `toJSON()` appelle `toISOString()`, ce qui produit des dates au format ISO 8601 en UTC.

```
class Montant {
  constructor(centimes, devise) {
    this.centimes = centimes;
    this.devise = devise;
  }

  // Appelee automatiquement par JSON.stringify
  toJSON() {
    return { valeur: (this.centimes / 100).toFixed(2), devise: this.devise };
  }
}

console.log(JSON.stringify({ prix: new Montant(4990, "EUR") }));
// {"prix":{"valeur":"49.90","devise":"EUR"}}

// Date possede deja une methode toJSON qui appelle toISOString
console.log(JSON.stringify({ le: new Date(Date.UTC(2026, 9, 8, 22)) }));
// {"le":"2026-10-08T22:00:00.000Z"}
```

Code 4 : Méthode `toJSON` d'une classe et d'une date (`exemples/js/02_tojson.mjs`)

4.3 Les erreurs de sérialisation

Deux situations font échouer `JSON.stringify`. Un `BigInt` provoque `TypeError: Do not know how to serialize a BigInt`. Une référence circulaire provoque aussi une `TypeError`, dont le message dépend du moteur : `Converting circular structure to JSON` dans V8 (Node.js, Chrome), `cyclic object value` dans Firefox. Pour sérialiser malgré tout une structure cyclique, une fonction `replacer` peut suivre la chaîne des objets parents et remplacer toute valeur qui pointe vers l'un d'eux.

```js
function safeStringify(obj, space) {
  const ancetres = [];
  return JSON.stringify(obj, function (cle, valeur) {
    if (typeof valeur !== "object" || valeur === null) return valeur;
    // "this" est l'objet qui contient la cle en cours :
    // on retire de la pile les objets qui ne sont plus des ancetres.
    while (ancetres.length > 0 && ancetres.at(-1) !== this) {
      ancetres.pop();
    }
    if (ancetres.includes(valeur)) return "[Reference circulaire]";
    ancetres.push(valeur);
    return valeur;
  }, space);
}

const noeud = { nom: "racine", enfant: {} };
noeud.enfant.parent = noeud; // cycle
console.log(safeStringify(noeud));
// {"nom":"racine","enfant":{"parent":"[Reference circulaire]"}}

const x = { n: 1 };
console.log(safeStringify({ a: x, b: x })); // objet partage, pas de cycle
// {"a":{"n":1},"b":{"n":1}}
```

Code 5 : Remplacer les références circulaires (`exemples/js/03_cycles.mjs`)

La fonction est déclarée avec `function` et non sous forme de fonction fléchée, car elle a besoin de `this`, qui désigne l'objet parent de la clé en cours. Une variante courante range tous les objets déjà vus dans un `WeakSet`. Elle produit des faux positifs : dans le second appel de l'exemple, l'objet `x` apparaît deux fois sans former de cycle, et cette variante aurait remplacé `b` par le marqueur.

4.4 Lire avec `JSON.parse`

`JSON.parse(texte, reviver)` construit la valeur JavaScript qui correspond au texte. La fonction `reviver`, facultative, est appelée pour chaque paire clé-valeur, des valeurs les plus profondes vers la racine, et peut remplacer la valeur. Elle sert surtout à recréer des types que JSON ne connaît pas, comme les dates.

```js
const texte = '{"nom":"Rapport","genereLe":"2026-10-08T22:00:00.000Z"}';

// Dates ISO 8601 en UTC uniquement (suffixe Z), millisecondes facultatives
const dateISO = /^\d{4}-\d{2}-\d{2}T\d{2}:\d{2}:\d{2}(\.\d{3})?Z$/;

const donnees = JSON.parse(texte, (cle, valeur) => {
  if (typeof valeur === "string" && dateISO.test(valeur)) {
    return new Date(valeur);
  }
  return valeur;
});

console.log(donnees.genereLe instanceof Date); // true
console.log(donnees.genereLe.getUTCFullYear()); // 2026
```

Code 6 : Recréer des objets `Date` avec un `reviver` (`exemples/js/04_reviver.mjs`)

L'expression régulière de l'exemple ne reconnaît que les dates en UTC, avec le suffixe `Z`. Une date avec décalage horaire, comme `2026-10-08T23:00:00+01:00`, resterait une chaîne. Convertir toute chaîne qui ressemble à une date est d'ailleurs risqué. Il vaut mieux convertir des champs précis, ou valider avec un schéma (section 4.7).

4.5 Gérer les erreurs de lecture

`JSON.parse` lève une `SyntaxError` dès que le texte n'est pas du JSON valide. Sans `try/catch`, cette erreur interrompt le programme. En pratique, le cas le plus courant n'est pas un JSON mal écrit, mais une réponse qui n'est pas du JSON du tout : une page HTML d'erreur renvoyée par un serveur ou un proxy, alors que le code appelle `reponse.json()`. Le message obtenu, du type `Unexpected token '<'`, cache alors la vraie cause. Vérifier le statut HTTP et l'en-tête `Content-Type` avant de lire le corps donne une erreur claire.

```javascript
function lireJSON(texte) {
  try {
    return { ok: true, valeur: JSON.parse(texte) };
  } catch (err) {
    if (err instanceof SyntaxError) return { ok: false, erreur: err.message };
    throw err; // toute autre erreur n'est pas un probleme de syntaxe
  }
}

console.log(lireJSON('{"a": 1}').ok);         // true
console.log(lireJSON("{'a': 1}").ok);         // false
console.log(lireJSON("<!DOCTYPE html>").ok);  // false

// Avec fetch : verifier le statut et le type de contenu avant .json()
async function lireReponse(reponse) {
  if (!reponse.ok) throw new Error(`HTTP ${reponse.status}`);
  const type = reponse.headers.get("content-type") ?? "";
  if (!type.includes("json")) {
    const debut = (await reponse.text()).slice(0, 40);
    throw new Error(`Reponse non JSON (${type}) : ${debut}`);
  }
  return reponse.json();
}

// Simulation d'un serveur qui renvoie une page HTML au lieu de JSON
const pageErreur = new Response("<!DOCTYPE html><h1>Maintenance</h1>", {
  headers: { "content-type": "text/html" },
});
try {
  await lireReponse(pageErreur); // en vrai : await lireReponse(await fetch(url))
} catch (err) {
  console.log(err.message);
  // Reponse non JSON (text/html) : <!DOCTYPE html><h1>Maintenance</h1>
}
```

Code 7 : Attraper les erreurs de syntaxe et les réponses qui ne sont pas du JSON (`exemples/js/05_erreurs_parse.mjs`)

Le test `type.includes("json")` accepte aussi les types spécialisés comme `application/problem+json` (section 10.5).

4.6 Copier un objet : JSON ou `structuredClone`

L'aller-retour `JSON.parse(JSON.stringify(obj))` sert souvent à copier un objet en profondeur. Il perd des informations sans prévenir : les dates deviennent des chaînes, les `Map` et `Set` deviennent des objets vides et les clés de valeur `undefined` disparaissent. La fonction `structuredClone`, disponible dans Node.js depuis la version 17 et dans les navigateurs récents, fait une vraie copie et conserve ces types.

```js
const original = {
  nom: "Rapport",
  creeLe: new Date("2026-10-08T22:00:00Z"),
  tags: new Set(["json", "manuel"]),
  note: undefined,
};

// Copie par aller-retour JSON : types perdus sans avertissement
const copieJSON = JSON.parse(JSON.stringify(original));
console.log(typeof copieJSON.creeLe); // string
console.log(copieJSON.tags);          // {}
console.log("note" in copieJSON);     // false

// structuredClone conserve Date, Set, Map et les valeurs undefined
const copie = structuredClone(original);
console.log(copie.creeLe instanceof Date);     // true
console.log(copie.tags.has("json"));           // true
console.log(copie.creeLe !== original.creeLe); // true : vraie copie
```

Code 8 : Copie par JSON et copie par `structuredClone` (`exemples/js/06_structuredclone.mjs`)

`structuredClone` a lui aussi des limites. Il lève une erreur sur les fonctions, et une instance de classe devient un objet simple, sans ses méthodes. Dans ces cas, il faut écrire une méthode de copie dans la classe.

4.7 Typer et valider en TypeScript

En TypeScript, `JSON.parse` renvoie le type `any`, qui désactive toute vérification. Un type récursif décrit exactement l'ensemble des valeurs JSON :

```ts
export type JsonPrimitive = string | number | boolean | null;
export type JsonObject = { [cle: string]: JsonValue };
export type JsonArray = JsonValue[];
export type JsonValue = JsonPrimitive | JsonObject | JsonArray;

// Le compilateur refuse ce qui n'est pas du JSON
const ok: JsonValue = { nom: "Ada", roles: ["admin"], actif: true, titre: null };
// const ko: JsonValue = { creeLe: new Date() }; // erreur : Date n'est pas JSON

console.log(JSON.stringify(ok));
```

Code 9 : Type récursif d'une valeur JSON (`exemples/js/07_types_json.ts`)

Ce type décrit la forme générale d'un JSON, pas le contenu attendu. Écrire `JSON.parse(texte) as Utilisateur` ne vérifie rien : le compilateur fait confiance, et une donnée mal formée circule jusqu'à ce qu'un champ manquant fasse échouer le code ailleurs. Les données externes doivent être validées à l'exécution. Avec Zod, le même schéma sert à valider et à déduire le type TypeScript.

```ts
import { z } from "zod";

const SchemaUtilisateur = z.object({
  id: z.number().int().positive(),
  email: z.email(),
  roles: z.array(z.string()).min(1),
  derniereConnexion: z.iso.datetime().optional(),
});

// Le type TypeScript est deduit du schema : une seule source de verite
type Utilisateur = z.infer<typeof SchemaUtilisateur>;

function lireUtilisateur(texte: string): Utilisateur {
  return SchemaUtilisateur.parse(JSON.parse(texte));
}

const ada = lireUtilisateur('{"id": 7, "email": "ada@example.com", "roles":
    ["admin"]}');
console.log(ada.email); // ada@example.com

const resultat = SchemaUtilisateur.safeParse({ id: -1, email: "ada", roles: [] });
if (!resultat.success) {
  console.log(resultat.error.issues.map((i) => i.path.join(".")));
  // [ 'id', 'email', 'roles' ]
}
```

Code 10 : Validation à l'exécution avec Zod 4 (`exemples/js/08_zod.ts`)

`parse` lève une erreur si les données sont invalides, alors que `safeParse` renvoie un objet qui contient soit les données, soit la liste des problèmes. Ce code suit la syntaxe de Zod 4. En Zod 3, on écrivait `z.string().email()` et `z.string().datetime()`, des formes encore acceptées mais dépréciées.

5 JSON en Python

5.1 Le module `json`

Le module `json` fait partie de la bibliothèque standard depuis Python 2.6. Il propose deux paires de fonctions : `dumps` et `loads` travaillent sur des chaînes, `dump` et `load` sur des fichiers. Le tableau 3 donne les correspondances de types.

Lecture (JSON vers Python)		**Écriture (Python vers JSON)**	
`object`	`dict`	`dict`	`object`
`array`	`list`	`list`, `tuple`	`array`
`string`	`str`	`str`	`string`
`number` entier	`int`	`int`	`number`
`number` décimal	`float`	`float`	`number`
`true`, `false`	`True`, `False`	`True`, `False`	`true`, `false`
`null`	`None`	`None`	`null`

Tableau 3 : Correspondance des types entre JSON et Python.

Deux conversions surprennent. Un tuple devient un tableau et revient sous forme de liste à la lecture. Les clés d'un dictionnaire sont converties en chaînes : `{1: "a"}` devient `{"1": "a"}`, et la relecture ne rend pas l'entier d'origine.

5.2 Les paramètres de `json.dumps`

```python
import json

donnees = {
    "utilisateur": "Hélène",
    "actif": True,
    "scores": [19.5, 18.0, 20.0],
    "meta": {"version": 2, "client": "web"},
}

# Lisible : indentation, accents conserves, cles triees
print(json.dumps(donnees, indent=2, ensure_ascii=False, sort_keys=True))

# Par defaut, les caracteres non ASCII sont echappes
print(json.dumps({"utilisateur": "Hélène"}))  # {"utilisateur": "H\u00e9l\u00e8ne"}

# Compact : aucun espace apres les separateurs
print(json.dumps(donnees["meta"], separators=(",", ":")))  #
    {"version":2,"client":"web"}

# NaN : interdit par la grammaire JSON, mais produit par defaut en Python
print(json.dumps(float("nan")))  # NaN
try:
    json.dumps(float("nan"), allow_nan=False)
except ValueError as erreur:
    print("Refuse :", erreur)
```

Code 11 : Paramètres principaux de `json.dumps` (`exemples/python/01_dumps.py`)

`ensure_ascii` vaut `True` par défaut et remplace chaque caractère non ASCII par une séquence `\uXXXX`. Avec `False`, le texte garde ses accents et devient plus léger : en UTF-8, é occupe 2 octets, contre 6 pour `\u00e9`.

`separators` vaut `(', ', ': ')` sans indentation et `(',', ': ')` avec indentation. La valeur `(',', ':')` supprime tous les espaces.

`sort_keys=True` trie les clés, ce qui rend la sortie stable, par exemple pour comparer deux fichiers. Le tri ne suffit pas à produire une forme canonique pour une signature numérique : l'écriture des nombres et l'échappement des caractères doivent aussi être fixés, ce que définit la RFC 8785 [19].

`allow_nan` vaut `True` par défaut. Python écrit alors `NaN` ou `Infinity`, que la grammaire JSON interdit et que `JSON.parse` refuse en JavaScript. Avec `allow_nan=False`, Python lève une `ValueError`. À la lecture, `json.loads` accepte ces valeurs par défaut, et le paramètre `parse_constant` permet de les refuser.

Fichiers sous Windows : Ouvrez toujours les fichiers JSON avec `encoding="utf-8"`. Sans ce paramètre, Python utilise l'encodage par défaut du système, souvent cp1252 sous Windows, et un `json.dump` avec `ensure_ascii=False` peut échouer sur certains caractères ou produire un fichier mal lu ailleurs.

5.3 Gérer les erreurs de lecture

`json.loads` et `json.load` lèvent `json.JSONDecodeError` quand le texte est invalide. Cette exception hérite de `ValueError` et donne la position de l'erreur dans ses attributs `lineno`, `colno` et `msg`.

```python
import json

texte = '{\n  "nom": "Ada",\n  "age": 36,\n}'  # virgule en trop

try:
    json.loads(texte)
except json.JSONDecodeError as erreur:
    # JSONDecodeError herite de ValueError et indique la position exacte
    print(f"Ligne {erreur.lineno}, colonne {erreur.colno} : {erreur.msg}")
```

Code 12 : Localiser une erreur de syntaxe (`exemples/python/02_erreurs.py`)

Avec Python 3.13, ce script affiche `Ligne 3, colonne 12 : Illegal trailing comma before end of object`. Les versions précédentes donnent un message moins précis pour la même erreur.

Un cas fréquent sous Windows : un fichier enregistré par Windows PowerShell 5.1 avec `-Encoding UTF8` commence par un BOM, et Python refuse de le lire avec le message `Unexpected UTF-8 BOM (decode using utf-8-sig)`. Ouvrir le fichier avec `encoding="utf-8-sig"` retire le BOM s'il est présent.

5.4 Types non pris en charge

Les dates, les `UUID`, les `Decimal` et les énumérations font échouer `json.dumps` avec une `TypeError`. Une sous-classe de `json.JSONEncoder` dont la méthode `default` convertit ces types règle le problème.

```python
import json
import uuid
from datetime import datetime, date
from decimal import Decimal
from enum import Enum


class Statut(Enum):
    PAYEE = "payee"


class EncodeurEtendu(json.JSONEncoder):
    # Appelee uniquement pour les objets que json ne sait pas serialiser
    def default(self, obj):
        if isinstance(obj, (datetime, date)):
            return obj.isoformat()
        if isinstance(obj, uuid.UUID):
            return str(obj)
        if isinstance(obj, Decimal):
            return str(obj)  # float(obj) perdrait de la precision
        if isinstance(obj, Enum):
            return obj.value
        return super().default(obj)


facture = {
    "id": uuid.UUID("6f1c2a9e-3b4d-4e5f-8a7b-1c2d3e4f5a6b"),
    "emiseLe": datetime(2026, 10, 8, 22, 0),
    "montant": Decimal("149.99"),
    "statut": Statut.PAYEE,
}
print(json.dumps(facture, cls=EncodeurEtendu))
```

Code 13 : Encodeur pour les dates, UUID, `Decimal` et énumérations (`exemples/python/03_encodeur.py`)

Le `Decimal` devient une chaîne et non un `float`, qui réintroduirait les erreurs d'arrondi que `Decimal` sert à éviter (section 10.2). Pour la lecture, le paramètre `object_hook` reçoit chaque objet décodé sous forme de dictionnaire et peut le remplacer, par exemple par une instance de classe.

```python
import json
from dataclasses import dataclass


@dataclass
class Coordonnees:
    lat: float
    lon: float


def vers_coordonnees(d: dict):
    # Appelee pour chaque objet JSON decode, des plus profonds aux plus externes
    if d.keys() == {"lat", "lon"}:
        return Coordonnees(**d)
    return d


texte = '{"ville": "Libreville", "point": {"lat": 0.4162, "lon": 9.4673}}'
resultat = json.loads(texte, object_hook=vers_coordonnees)
print(resultat["point"])  # Coordonnees(lat=0.4162, lon=9.4673)
```

Code 14 : Recréer une dataclass avec `object_hook` (`exemples/python/04_object_hook.py`)

5.5 Valider avec Pydantic

Pydantic 2 valide et convertit les données à partir de classes annotées. Son cœur, `pydantic-core`, est écrit en Rust. `model_validate_json` analyse et valide le texte en une seule étape, et `model_dump_json` produit le JSON. C'est la bibliothèque sur laquelle repose FastAPI.

```python
from pydantic import BaseModel, EmailStr, Field, ValidationError


class Utilisateur(BaseModel):
    id: int = Field(gt=0)
    nom: str
    email: EmailStr
    tags: list[str] = []
    bloque: bool = False


texte = '{"id": 42, "nom": "Bob", "email": "bob@example.com"}'
bob = Utilisateur.model_validate_json(texte)  # analyse et validation en une passe
print(bob.model_dump_json())
# {"id":42,"nom":"Bob","email":"bob@example.com","tags":[],"bloque":false}

try:
    Utilisateur.model_validate_json('{"id": 0, "nom": "Bob", "email": "bob"}')
except ValidationError as erreur:
    print(erreur.error_count(), "erreurs :", [e["loc"][0] for e in
    erreur.errors()])
    # 2 erreurs : ['id', 'email']
```

Code 15 : Validation avec Pydantic 2 (`exemples/python/05_pydantic.py`)

Le type `EmailStr` nécessite le paquet `email-validator`, installé avec `pip install "pydantic[email]"`.

5.6 Bibliothèques plus rapides

Pour de gros volumes, deux bibliothèques compilées remplacent le module standard. **orjson**, écrite en Rust, sérialise directement les dates, les UUID et les dataclasses, ainsi que les tableaux `numpy` avec l'option `OPT_SERIALIZE_NUMPY`. Attention : `orjson.dumps` renvoie des `bytes`, pas une `str`. **msgspec** décode et valide en une seule passe vers des classes typées (`msgspec.Struct`). Leurs auteurs publient des mesures de performance, mais le gain réel dépend de vos données : mesurez-le avec le module `timeit` avant de changer de bibliothèque.

6 Valider avec JSON Schema

6.1 Rôle

La grammaire JSON ne dit rien du contenu attendu : n'importe quel objet est valide. JSON Schema décrit ce contenu (champs obligatoires, types, formats, bornes) dans un document qui est lui-même du JSON. La version actuelle, le draft 2020-12 [21], est publiée par le projet json-schema.org. OpenAPI, le format le plus répandu pour décrire les API REST, utilise JSON Schema pour décrire les corps de requête et de réponse. Depuis OpenAPI 3.1, les deux sont entièrement compatibles.

6.2 Un schéma complet

```json
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://example.com/schemas/produit.json",
  "title": "Produit",
  "type": "object",
  "properties": {
    "sku": { "type": "string", "pattern": "^[A-Z]{3}-[0-9]{4}$" },
    "nom": { "type": "string", "minLength": 3, "maxLength": 100 },
    "prix": { "type": "number", "exclusiveMinimum": 0 },
    "categories": {
      "type": "array",
      "items": { "type": "string" },
      "minItems": 1,
      "uniqueItems": true
    },
    "statut": { "enum": ["brouillon", "publie", "archive"] },
    "creeLe": { "type": "string", "format": "date-time" }
  },
  "required": ["sku", "nom", "prix", "categories"],
  "additionalProperties": false
}
```

Code 16 : Schéma d'un produit (draft 2020-12) (`exemples/json/produit.schema.json`)

`$schema` indique la version de la spécification, et `$id` donne au schéma une adresse qui permet de le référencer. `required` liste les champs obligatoires. `pattern` impose une expression régulière et `exclusiveMinimum` une borne stricte. `"additionalProperties": false` refuse tout champ non déclaré. Cette dernière règle protège contre l'affectation de masse (*mass assignment*) : un client qui ajoute `"role": "admin"` à sa requête est rejeté, au lieu de voir ce champ recopié en base.

Le mot-clé format n'est pas vérifié par défaut : En draft 2020-12, `format` est une simple annotation. Il faut activer sa vérification : avec le paquet `ajv-formats` pour AJV, avec `FORMAT_CHECKER` pour la bibliothèque Python `jsonschema`. En Python, le format `date-time` exige en plus le paquet `rfc3339-validator`. Sans lui, la vérification est ignorée sans aucun message.

6.3 Combiner et réutiliser

`allOf` exige que la valeur respecte tous les sous-schémas, `anyOf` au moins un, `oneOf` exactement un, et `not` refuse les valeurs qui respectent le sous-schéma. Les définitions réutilisables se placent dans `$defs` et s'utilisent avec `"$ref": "#/$defs/Adresse"`.

6.4 Valider en Python et en JavaScript

En Python, la bibliothèque `jsonschema` fournit un validateur par version de la spécification. `iter_errors` liste toutes les erreurs au lieu de s'arrêter à la première.

```python
import json
from jsonschema import Draft202012Validator

with open("json/produit.schema.json", encoding="utf-8") as f:
    schema = json.load(f)

# FORMAT_CHECKER active la verification de "format" (desactivee par defaut)
validateur = Draft202012Validator(
    schema, format_checker=Draft202012Validator.FORMAT_CHECKER
)

with open("json/produit.json", encoding="utf-8") as f:
    produit = json.load(f)
print(validateur.is_valid(produit))  # True

invalide = {**produit, "prix": 0, "creeLe": "hier", "couleur": "bleu"}
for erreur in validateur.iter_errors(invalide):
    print(list(erreur.path), erreur.message)
```

Code 17 : Validation avec `jsonschema` (`exemples/python/06_jsonschema.py`)

En JavaScript, AJV compile chaque schéma en une fonction de validation. Attention à l'import : la classe par défaut `Ajv` cible le draft-07, et un schéma qui déclare le draft 2020-12 exige la classe `Ajv2020`.

```js
import { readFileSync } from "node:fs";
import Ajv2020 from "ajv/dist/2020.js";
import addFormats from "ajv-formats";

const ajv = new Ajv2020({ allErrors: true });
addFormats(ajv); // sans cette ligne, "format" n'est pas verifie

const schema = JSON.parse(readFileSync("json/produit.schema.json", "utf8"));
const valider = ajv.compile(schema);

const produit = JSON.parse(readFileSync("json/produit.json", "utf8"));
console.log(valider(produit)); // true

const invalide = { ...produit, prix: 0, creeLe: "hier", couleur: "bleu" };
console.log(valider(invalide)); // false
for (const e of valider.errors) {
  console.log(e.instancePath || "(racine)", e.message);
}
```

Code 18 : Validation avec AJV et `ajv-formats` (`exemples/js/10_ajv.mjs`)

6.5 Générer le schéma depuis le code

Écrire un schéma à la main en plus des classes crée deux sources à garder synchronisées. Zod 4 et Pydantic produisent le schéma à partir du modèle : `z.toJSONSchema(schema)` d'un côté, `Modele.model_json_schema()` de l'autre (code 20).

```js
import { z } from "zod";

const Produit = z.object({
  sku: z.string().regex(/^[A-Z]{3}-[0-9]{4}$/),
  prix: z.number().positive(),
});

// Zod 4 produit directement le JSON Schema correspondant
console.log(JSON.stringify(z.toJSONSchema(Produit), null, 2));
```

Code 19 : JSON Schema généré par Zod 4 (`exemples/js/09_zod_vers_schema.ts`)

6.6 Contraindre la réponse d'un modèle de langage

Les modèles de langage servent de plus en plus à extraire des données structurées d'un texte : une facture, un formulaire, un courriel. Plusieurs API acceptent un JSON Schema pour contraindre la forme de la réponse. Ollama, qui fait tourner gratuitement des modèles sur votre propre machine, le prend en charge depuis sa version 0.5 avec le champ `format`. Les API d'OpenAI et d'Anthropic proposent aussi des sorties structurées, chacune avec ses restrictions sur les mots-clés acceptés.

```python
import json

from pydantic import BaseModel, Field


class Facture(BaseModel):
    fournisseur: str
    montant_total: float = Field(ge=0)
    devise: str = Field(pattern=r"^[A-Z]{3}$")
    lignes: list[str]


# 1. Le JSON Schema est genere a partir du modele Python
schema = Facture.model_json_schema()
print(json.dumps(schema["required"]))
# ["fournisseur", "montant_total", "devise", "lignes"]

# 2. Corps de requete pour un modele local servi par Ollama (POST /api/chat)
requete = {
    "model": "llama3.2",
    "messages": [{"role": "user", "content": "Extrais la facture suivante : ..."}],
    "format": schema,  # la reponse devra respecter ce schema
    "stream": False,
}

# 3. La reponse se valide comme n'importe quelle donnee externe
reponse = '{"fournisseur": "Papeterie du Centre", "montant_total": 15000,
    "devise": "XAF", "lignes": ["2 cahiers", "1 stylo"]}'
facture = Facture.model_validate_json(reponse)
print(facture.montant_total, facture.devise)  # 15000.0 XAF
```

Code 20 : Schéma généré par Pydantic pour une extraction de données (`exemples/python/09_schema_llm.py`)

Même contrainte par un schéma, la réponse doit être validée. Elle peut être tronquée si la limite de longueur est atteinte, et un champ peut avoir la bonne forme mais une valeur fausse. Le même modèle Pydantic sert à générer le schéma et à valider la réponse.

7 Formats et standards dérivés

7.1 JSON Pointer (RFC 6901)

JSON Pointer [5] désigne une valeur précise d'un document par un chemin de segments séparés par `/`. Les indices de tableau commencent à 0. Dans les noms de clés, `~` s'écrit `~0` et `/` s'écrit `~1`.

```
{
  "utilisateurs": [
    { "nom": "Alice", "contacts": { "email/pro": "alice@example.com" } }
  ]
}
```

Code 21 : Document utilisé pour les exemples de JSON Pointer (`exemples/json/contacts.json`)

Dans ce document, `/utilisateurs/0/nom` désigne `"Alice"`, et `/utilisateurs/0/contacts/email~1pro` désigne l'adresse associée à la clé `"email/pro"`. JSON Patch et le mot-clé `$ref` de JSON Schema utilisent cette syntaxe.

7.2 JSON Patch (RFC 6902) et JSON Merge Patch (RFC 7396)

Ces deux formats décrivent une modification partielle, généralement envoyée avec la méthode HTTP `PATCH`. JSON Patch [4] est une liste d'opérations appliquées dans l'ordre : `add`, `remove`, `replace`, `move`, `copy` et `test`. L'application est atomique : si une opération échoue, le document reste inchangé. L'opération `test` sert de garde-fou, par exemple pour ne valider une commande que si elle est encore en attente.

```
{
  "statut": "en_attente",
  "montant": 15000,
  "brouillon": { "note": "a verifier" }
}
```

Code 22 : Document de départ (`exemples/json/commande.json`)

```
[
  { "op": "test", "path": "/statut", "value": "en_attente" },
  { "op": "replace", "path": "/statut", "value": "validee" },
  { "op": "add", "path": "/validePar", "value": "Administrateur" },
  { "op": "remove", "path": "/brouillon" }
]
```

Code 23 : JSON Patch appliqué au document (`exemples/json/commande.patch.json`)

Ce patch produit `{"statut": "validee", "montant": 15000, "validePar":`

`"Administrateur"}`. Si le statut n'est plus `"en_attente"`, l'opération `test` échoue et rien n'est modifié.

JSON Merge Patch [11] est plus simple : on envoie un objet qui contient seulement les champs à changer, et `null` supprime un champ. Le patch `{"statut": "validee", "brouillon": null}` produit le même effet que l'exemple précédent, sans l'ajout de `validePar`. Ce format a deux limites : un tableau est toujours remplacé en entier, et il est impossible de donner la valeur `null` à un champ.

7.3 JSONPath (RFC 9535)

JSONPath sélectionne des valeurs dans un document avec une syntaxe proche de XPath. Proposé par Stefan Gössner en 2007, longtemps implémenté de façon incohérente, il est normalisé depuis février 2024 par la RFC 9535 [10]. `$` désigne la racine, `..` cherche à toutes les profondeurs, `[*]` prend tous les éléments et `[?...]` filtre, avec `@` pour l'élément en cours. Appliquées au fichier `json/utilisateurs.json`, ces requêtes donnent :

```
$.utilisateurs[*].nom                          tous les noms
$..nom                                         toutes les cles "nom", a toute
    profondeur
$.utilisateurs[?@.statut == 'actif'].nom       noms des comptes actifs
$.utilisateurs[0]                              premier utilisateur
```

Code 24 : Requêtes JSONPath

L'outil `jq` (section 12) utilise une autre syntaxe mais couvre les mêmes besoins en ligne de commande.

7.4 JSON Lines

JSON Lines (extension `.jsonl`, aussi appelé NDJSON) place un document JSON complet par ligne. Un tableau JSON de plusieurs gigaoctets n'est valide qu'une fois son crochet fermant lu, alors qu'un fichier JSON Lines se traite ligne par ligne, avec une mémoire constante. On ajoute un enregistrement en écrivant une ligne à la fin du fichier, sans le relire, et un fichier coupé en morceaux reste exploitable. C'est le format courant des journaux applicatifs, des exports de données et des jeux de données d'entraînement et d'évaluation en apprentissage automatique.

```
{"capteur": "A", "valeur": 21.5, "heure": "08:00"}
{"capteur": "B", "valeur": 19.0, "heure": "08:00"}
{"capteur": "A", "valeur": 23.5, "heure": "09:00"}
{"capteur": "B", "valeur": 20.0, "heure": "09:00"}
```

Code 25 : Fichier JSON Lines (`exemples/json/mesures.jsonl`)

```python
import json

import pandas as pd

# Lecture ligne par ligne : chaque ligne est un document JSON complet
with open("json/mesures.jsonl", encoding="utf-8") as f:
    mesures = [json.loads(ligne) for ligne in f if ligne.strip()]
print(len(mesures), "mesures")  # 4 mesures

# pandas lit directement le JSON Lines dans un DataFrame
df = pd.read_json("json/mesures.jsonl", lines=True)
print(df.groupby("capteur")["valeur"].mean().to_dict())  # {'A': 22.5, 'B': 19.5}

# Ajouter un enregistrement = ajouter une ligne (mode "a"), sans relire le fichier :
# with open("mesures.jsonl", "a", encoding="utf-8") as f:
#     f.write(json.dumps({"capteur": "A", "valeur": 22.0}) + "\n")
```

Code 26 : Lire un fichier JSON Lines en Python et avec pandas (`exemples/python/08_jsonl.py`)

7.5 JSON-LD et JWT

JSON-LD [15] ajoute du sens aux données avec les clés `@context`, `@id` et `@type`, qui relient chaque propriété à un vocabulaire partagé. Son usage principal est le référencement : les moteurs de recherche lisent le vocabulaire Schema.org écrit en JSON-LD dans les pages web.

Un JSON Web Token (JWT, RFC 7519 [14]) est formé de trois segments encodés en Base64URL et séparés par des points : un en-tête et une charge utile en JSON, puis une signature. Encodé ne veut pas dire chiffré : n'importe qui peut lire la charge utile, sans clé.

```python
import base64
import json

# en-tete . charge utile . signature (jeton de demonstration)
jeton = (
    "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9"
    ".eyJzdWIiOiI0MiIsInJvbGUiOiJhZG1pbiIsImV4cCI6MTc5MTUwMDAwMH0"
    ".7G4nli0DKe1i4y7RFj9X7dYdcuCgXs4a5yftUsM9cvU"
)


def decoder(segment: str) -> dict:
    remplissage = "=" * (-len(segment) % 4)  # Base64URL omet les "="
    return json.loads(base64.urlsafe_b64decode(segment + remplissage))


entete, charge_utile, _signature = jeton.split(".")
print(decoder(entete))        # {'alg': 'HS256', 'typ': 'JWT'}
print(decoder(charge_utile))  # {'sub': '42', 'role': 'admin', 'exp': 1791500000}
# Aucune cle n'a ete necessaire : la charge utile est lisible par tous.
```

Code 27 : Lire la charge utile d'un JWT sans aucune clé (`exemples/python/10_jwt.py`)

Deux règles en découlent. Ne placez jamais de secret dans la charge utile d'un JWT signé. Et vérifiez toujours la signature côté serveur avec une bibliothèque reconnue (PyJWT en Python, `jose` en JavaScript) en fixant la liste des algorithmes acceptés : un serveur qui se fie à l'algorithme annoncé dans l'en-tête, y compris `none`, accepte des jetons forgés. La RFC 8725 [20] rassemble ces bonnes pratiques.

8 Stocker du JSON dans PostgreSQL

8.1 `json` ou `jsonb`

PostgreSQL propose deux types. `json` stocke le texte tel quel : espaces, ordre des clés et clés en double sont conservés, et le texte est réanalysé à chaque requête. `jsonb` stocke une forme binaire décomposée : l'écriture est un peu plus lente, mais les requêtes sont plus rapides et le type s'indexe. En contrepartie, `jsonb` normalise le document : il supprime les espaces, réordonne les clés et ne garde que la dernière valeur d'une clé en double. Sauf besoin de conserver le texte exact, pour archiver une charge utile signée par exemple, choisissez `jsonb`.

8.2 Requêtes essentielles

Le tableau 4 résume les opérateurs utiles, et le code 28 les met en pratique.

Opérateur	Rôle	Exemple
`->`	Valeur d'une clé ou d'un indice, en `jsonb`	`details -> 'livraison'`
`->>`	Même valeur, convertie en `text`	`details ->> 'express'`
`#>`, `#>>`	Valeur au bout d'un chemin	`details #>> '{livraison,ville}'`
`@>`	Le document contient-il ce fragment ?	`details @> '{"express": true}'`
`?`	La clé existe-t-elle ?	`details ? 'express'`
`jsonb_set`	Remplace la valeur d'un chemin	voir le code 28

Tableau 4 : Opérateurs et fonctions `jsonb` courants.

```sql
-- Schema dedie aux exemples, pour ne rien melanger avec vos propres tables
CREATE SCHEMA IF NOT EXISTS manuel_json;
SET search_path TO manuel_json;

CREATE TABLE commandes (
  id      bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  client  text  NOT NULL,
  details jsonb NOT NULL
);

INSERT INTO commandes (client, details) VALUES
  ('Ada',   '{"livraison": {"ville": "Libreville"}, "articles": ["cahier",
    "stylo"], "express": true}'),
  ('Grace', '{"livraison": {"ville": "Douala"}, "articles": ["cartable"],
    "express": false}');

-- -> renvoie du jsonb, ->> renvoie du texte
SELECT client,
       details -> 'livraison' ->> 'ville'        AS ville,
       jsonb_array_length(details -> 'articles') AS nb_articles
FROM commandes;

-- @> teste si le document contient un fragment ; un index GIN l'accelere
CREATE INDEX commandes_details_idx ON commandes USING GIN (details);
SELECT client FROM commandes WHERE details @> '{"express": true}';

-- Modifier une valeur imbriquee sans reecrire le reste du document
UPDATE commandes
SET details = jsonb_set(details, '{livraison,ville}', '"Port-Gentil"')
WHERE client = 'Ada';
SELECT details -> 'livraison' ->> 'ville' AS ville FROM commandes WHERE client =
    'Ada';

-- json garde le texte tel quel ; jsonb le normalise (derniere cle dupliquee
    gardee)
SELECT '{"b": 1, "a": 2, "a": 3}'::json  AS texte_json,
       '{"b": 1, "a": 2, "a": 3}'::jsonb AS texte_jsonb;
```

Code 28 : Créer, interroger, indexer et modifier une colonne `jsonb` (`exemples/sql/jsonb.sql`)

La première requête renvoie `Libreville` et `Douala` avec le nombre d'articles, la deuxième seulement `Ada`, et la dernière montre la normalisation : `{"b": 1, "a": 2, "a": 3}` pour `json`, `{"a": 3, "b": 1}` pour `jsonb`.

L'index GIN créé dans l'exemple accélère `@>` et `?`. Sans index, PostgreSQL doit lire chaque ligne et analyser chaque document.

8.3 Quand utiliser `jsonb`

`jsonb` convient aux données dont la forme varie : attributs différents selon le type de produit, préférences d'un utilisateur, charge utile brute d'un webhook conservée pour audit. Les champs que vous filtrez, joignez ou contraignez souvent doivent rester des colonnes classiques : une colonne a un type vérifié, accepte une clé étrangère et dispose

de statistiques que l'optimiseur de requêtes utilise. Un compromis courant consiste à extraire en colonnes les quelques champs importants et à garder le reste en `jsonb`.

Supabase repose sur PostgreSQL, tout ce qui précède s'y applique donc. Son client JavaScript accepte la même notation dans les filtres, par exemple `.eq("details->livraison->>ville","Libreville")`, et l'extension `pg_jsonschema` permet de valider une colonne contre un JSON Schema dans une contrainte `CHECK`.

9 Gros volumes et performance

9.1 Le coût d'un chargement complet

`JSON.parse` et `json.load` lisent tout le texte, puis construisent tous les objets en mémoire. L'arbre obtenu occupe en général plusieurs fois la taille du fichier, à cause des tables de hachage des dictionnaires, des pointeurs et des en-têtes de chaque objet. Le facteur dépend du langage et des données, et se mesure, par exemple avec le module `tracemalloc` en Python. Sur un serveur ou un conteneur dont la mémoire est limitée, un fichier de quelques centaines de mégaoctets peut suffire à faire arrêter le processus pour dépassement de mémoire.

9.2 Lire par flux

Un parseur par flux lit le texte morceau par morceau et rend les éléments un à un, sans construire l'arbre complet. En Python, `ijson` renvoie les éléments d'un tableau désigné par un préfixe : `"utilisateurs.item"` désigne chaque élément du tableau `utilisateurs`.

```python
import ijson

# ijson lit le fichier par morceaux : un seul utilisateur en memoire a la fois
with open("json/utilisateurs.json", "rb") as f:
    actifs = sum(
        1 for u in ijson.items(f, "utilisateurs.item") if u["statut"] == "actif"
    )

print(f"Comptes actifs : {actifs}")  # Comptes actifs : 3
```

Code 29 : Compter des éléments sans charger tout le fichier (`exemples/python/07_ijson.py`)

La mémoire utilisée dépend alors de la taille d'un élément, pas de celle du fichier. En Node.js, le paquet `stream-json` joue le même rôle avec les flux. Quand vous choisissez le format, JSON Lines (section 7.4) évite le problème à la source.

9.3 Compression

Les noms de clés se répètent à chaque enregistrement, ce qui rend JSON très compressible. Activer la compression HTTP (Gzip ou Brotli) sur le serveur réduit fortement la taille transférée, et Brotli compresse en général mieux que Gzip sur du texte. La minification, qui supprime les espaces, réduit aussi la taille brute, mais son gain devient faible une fois la compression activée. Dans les deux cas, mesurez sur vos propres réponses.

10 Sécurité

10.1 Imbrication profonde et documents géants

Un document de quelques dizaines de kilo-octets peut contenir des dizaines de milliers de crochets imbriqués. Un parseur récursif consomme de la pile d'appels à chaque niveau, jusqu'à l'épuiser. En Python, `json.loads` lève alors une `RecursionError` ; selon le langage, le résultat va de l'exception au plantage du processus. Un document simplement énorme, lui, sature la mémoire.

Deux limites à fixer : Limitez la taille du corps des requêtes, en l'adaptant à votre API : Express, par exemple, refuse par défaut les corps JSON de plus de 100 ko. Limitez aussi la profondeur d'imbrication acceptée : 64 niveaux est une valeur courante, c'est celle de `System.Text.Json` en .NET.

10.2 Les grands entiers

JavaScript représente tous les nombres en double précision IEEE 754 [12], avec 53 bits de précision. Les entiers ne sont exacts que dans l'intervalle

$$[-(2^{53}-1),\ 2^{53}-1], \qquad 2^{53}-1 = 9\,007\,199\,254\,740\,991. \tag{1}$$

La RFC 8259 (section 6) cite elle-même cet intervalle comme celui sur lequel les implémentations s'accordent. Au-delà, `JSON.parse` arrondit sans prévenir. C'est le cas des clés `BIGINT` des bases de données et des identifiants distribués de type Snowflake, utilisés par exemple par Twitter et Discord.

```
const texte = '{"id": 9007199254740993, "idTexte": "9007199254740993"}';
const donnees = JSON.parse(texte);

console.log(donnees.id);                        // 9007199254740992 : valeur faussee
console.log(Number.isSafeInteger(donnees.id)); // false
console.log(BigInt(donnees.idTexte));           // 9007199254740993n : intacte
```

Code 30 : Un identifiant faussé par `JSON.parse` (`exemples/js/11_precision.mjs`)

Règle : Envoyez les identifiants et les entiers sur 64 bits sous forme de chaînes. Côté JavaScript, convertissez-les avec `BigInt` seulement si un calcul est nécessaire.

10.3 Clés dupliquées

La RFC 8259 n'impose pas de rejeter les clés en double. Prenons `{"role": "lecteur", "role": "admin"}`. Si un pare-feu applicatif garde la première valeur et laisse passer

la requête, et que le serveur placé derrière lui garde la dernière, l'attaquant obtient le rôle `admin` sans que le contrôle l'ait vu. `JSON.parse`, `json.loads` et `jsonb` gardent tous la dernière valeur. La RFC 7493 (I-JSON) [2] interdit les doublons : c'est le comportement à exiger des composants qui contrôlent l'accès. En Python, le paramètre `object_pairs_hook` permet de les refuser.

```python
import json

texte = '{"role": "lecteur", "role": "admin"}'
print(json.loads(texte))  # {'role': 'admin'} : la derniere valeur gagne


def sans_doublons(paires: list[tuple]) -> dict:
    # Recoit les paires (cle, valeur) dans l'ordre, doublons compris
    cles = [cle for cle, _ in paires]
    doublons = sorted({c for c in cles if cles.count(c) > 1})
    if doublons:
        raise ValueError(f"Cles en double : {doublons}")
    return dict(paires)


try:
    json.loads(texte, object_pairs_hook=sans_doublons)
except ValueError as erreur:
    print(erreur)  # Cles en double : ['role']
```

Code 31 : Refuser les clés dupliquées en Python (`exemples/python/11_doublons.py`)

10.4 Pollution de prototype en JavaScript

`JSON.parse` crée la clé `__proto__` comme une propriété ordinaire : le parseur lui-même ne pollue rien. Le danger vient d'une fonction de fusion récursive qui recopie ensuite cette clé dans un objet existant. Elle modifie alors le prototype commun à tous les objets, et une propriété comme `admin` apparaît sur chacun d'eux.

```javascript
const INTERDITES = new Set(["__proto__", "constructor", "prototype"]);

// Fusion recursive ; avec protegee = false, elle reproduit l'erreur courante
function fusion(cible, source, protegee) {
  for (const cle of Object.keys(source)) {
    if (protegee && INTERDITES.has(cle)) continue;
    if (typeof source[cle] === "object" && source[cle] !== null) {
      cible[cle] ??= {};
      fusion(cible[cle], source[cle], protegee);
    } else {
      cible[cle] = source[cle];
    }
  }
  return cible;
}

const entree = JSON.parse('{"__proto__": {"admin": true}}');
console.log(Object.keys(entree)); // [ '__proto__' ] : simple propriete

fusion({}, entree, false);
console.log({}.admin); // true : le prototype de TOUS les objets est modifie
delete Object.prototype.admin; // nettoyage pour la suite de l'exemple

fusion({}, entree, true);
console.log({}.admin); // undefined : les cles dangereuses sont ignorees
```

Code 32 : Fusion récursive avec et sans protection (`exemples/js/12_pollution.mjs`)

Pour s'en protéger, les fonctions de fusion doivent ignorer `__proto__`, `constructor` et `prototype`. Une `Map` ou un objet créé avec `Object.create(null)` n'a pas de prototype à polluer, et un schéma qui refuse les champs inconnus bloque ces clés dès l'entrée.

10.5 Format des erreurs d'API (RFC 9457)

La RFC 9457 [18], qui remplace depuis 2023 la RFC 7807 [17], définit un format standard pour décrire une erreur d'API, avec le type de média `application/problem+json`. `type` identifie le genre d'erreur par une URI, `title` le résume, `status` reprend le code HTTP, `detail` explique ce cas précis et `instance` désigne la ressource concernée.

```json
{
  "type": "https://example.com/erreurs/solde-insuffisant",
  "title": "Solde insuffisant",
  "status": 403,
  "detail": "Le solde est de 3000 XAF, le retrait demande est de 5000 XAF.",
  "instance": "/comptes/12345/retraits/abc"
}
```

Code 33 : Erreur d'API au format RFC 9457 (`exemples/json/probleme.json`)

11 Choisir un format

11.1 Comparaison

Le tableau 5 compare JSON à quatre autres formats sur des critères vérifiables. Le chapitre 4 de *Designing Data-Intensive Applications* [16] discute en détail les compromis entre formats textuels et binaires, ainsi que l'évolution des schémas.

	JSON	**YAML**	**XML**	**Protobuf**	**MessagePack**
Encodage	Texte	Texte	Texte	Binaire	Binaire
Lisible sans outil	Oui	Oui	Oui	Non	Non
Commentaires	Non	Oui	Oui	Non[a]	Non
Schéma	Facultatif (JSON Schema)	Facultatif	Facultatif (XSD)	Obligatoire (`.proto`)	Facultatif
Données binaires	En Base64	En Base64	En Base64	Natives	Natives

Tableau 5 : Comparaison de formats d'échange. [a] Les commentaires vont dans le fichier `.proto`, pas dans les données.

11.2 Recommandations

JSON reste le bon choix par défaut pour les API publiques et les échanges avec un navigateur. Pour un fichier de configuration écrit à la main, YAML ou une variante tolérante de JSON comme JSONC (avec commentaires, utilisée par VS Code) ou JSON5 sont plus confortables. Pour des échanges internes à fort débit entre services, Protocol Buffers avec gRPC réduit la taille des messages et impose un contrat. Pour réduire la taille sans changer la structure des données, MessagePack ou CBOR (RFC 8949) encodent en binaire un modèle de données proche de celui de JSON.

12 Outils en ligne de commande

12.1 jq

`jq` lit, filtre et transforme du JSON dans un terminal. Sous Windows, il s'installe avec `winget install jqlang.jq`. Les commandes suivantes se lancent depuis le dossier `exemples/`, et les tests vérifient leurs résultats.

```
jq . json/utilisateur.json                  # formate le document
jq .profil.prenom json/utilisateur.json     # "Ada"
jq -r '.roles[]' json/utilisateur.json      # une valeur par ligne, sans guillemets
jq -c .coordonnees json/utilisateur.json    # sortie compacte sur une ligne
jq -s length json/mesures.jsonl             # -s regroupe les lignes : 4
jq '.utilisateurs[] | select(.statut == "actif") | .nom' json/utilisateurs.json
```

Code 34 : Commandes jq courantes

Windows PowerShell 5.1 supprime les guillemets doubles placés à l'intérieur d'un argument quand il appelle un programme externe. La dernière commande ne fonctionne donc qu'avec PowerShell 7.3 ou plus récent. Avec la version 5.1, écrivez le filtre dans un fichier et passez-le avec `jq -f filtre.jq`.

12.2 Valider un fichier avec Python

`python -m json.tool fichier.json` valide un fichier et l'affiche formaté, sans rien installer. En cas d'erreur, la commande indique la ligne et la colonne. L'option `--sort-keys` trie les clés, `--compact` compacte la sortie et `--json-lines` traite un fichier JSON Lines.

12.3 PowerShell

PowerShell convertit le JSON en objets avec `ConvertFrom-Json`, et des objets en JSON avec `ConvertTo-Json`. L'option `-Raw` de `Get-Content` lit le fichier en une seule chaîne.

```
$u = Get-Content .\json\utilisateur.json -Raw | ConvertFrom-Json
$u.profil.prenom              # Ada
$u.roles -join ", "           # chercheuse, pionniere, admin
$u | ConvertTo-Json -Depth 10 | Set-Content sortie.json -Encoding utf8
```

Code 35 : Lire et écrire du JSON en PowerShell

`ConvertTo-Json` s'arrête par défaut à une profondeur de 2 et remplace les niveaux plus profonds par leur représentation textuelle, comme `"System.Collections.Hashtable"` : précisez toujours `-Depth`. Windows PowerShell 5.1 ajoute aussi un BOM avec `-Encoding UTF8`, que la RFC 8259 interdit et que Python refuse ; PowerShell 7 écrit de l'UTF-8 sans BOM.

13 Conclusion

JSON est simple à écrire, et la plupart des problèmes viennent de ce que sa grammaire ne dit pas : la précision des nombres, les clés en double, la forme attendue des données, la taille des documents.

Quelques règles couvrent l'essentiel. Validez toute donnée qui entre dans votre programme avec un schéma, en Python comme en TypeScript, et générez ce schéma à partir du modèle plutôt que de l'écrire deux fois. Envoyez les identifiants et les entiers sur 64 bits sous forme de chaînes. Refusez les clés dupliquées dans les composants qui contrôlent l'accès, et limitez la taille et la profondeur des documents reçus. Traitez explicitement les erreurs de lecture, en vérifiant le type de contenu avant d'appeler `.json()`. Pour les gros volumes, préférez JSON Lines et la lecture par flux. En base de données, utilisez `jsonb` pour les données de forme variable et gardez en colonnes les champs que vous interrogez souvent.

Pour aller plus loin, la RFC 8259 reste la référence, courte et lisible. Le guide *Understanding JSON Schema*, sur json-schema.org, explique les schémas pas à pas. Enfin, chaque exemple de ce manuel peut être modifié puis relancé avec sa suite de tests, ce qui permet de vérifier rapidement une hypothèse.

14 Fiche de référence

14.1 Règles de syntaxe

Les clés sont entre guillemets doubles droits. Pas de virgule après le dernier élément, pas de commentaire. `true`, `false` et `null` s'écrivent en minuscules. Pas de zéro en tête, pas de signe plus, pas de `NaN` ni d'`Infinity`. Le texte échangé est en UTF-8, sans BOM. Recommandé par les RFC 8259 et 7493 : des clés uniques, et les entiers au-delà de $2^{53} - 1$ envoyés sous forme de chaînes. Les tableaux 6 à 8 rassemblent les séquences d'échappement, les fonctions équivalentes en JavaScript et en Python, et les types de média.

Séquence	Caractère	Code
`\"`	Guillemet double	U+0022
`\\`	Barre oblique inversée	U+005C
`\/`	Barre oblique (échappement facultatif)	U+002F
`\b`	Retour arrière	U+0008
`\f`	Saut de page	U+000C
`\n`	Saut de ligne	U+000A
`\r`	Retour chariot	U+000D
`\t`	Tabulation	U+0009
`\uXXXX`	Caractère désigné par son code UTF-16	U+XXXX

Tableau 6 : Séquences d'échappement autorisées dans une chaîne JSON.

Opération	JavaScript	Python
Texte vers valeur	`JSON.parse(texte)`	`json.loads(texte)`
Fichier vers valeur	`JSON.parse(fs.readFileSync(f, "utf8"))`	`json.load(f)`
Valeur vers texte	`JSON.stringify(v)`	`json.dumps(v)`
Valeur vers fichier	`fs.writeFileSync(f, JSON.stringify(v))`	`json.dump(v, f)`
Indenter	`JSON.stringify(v, null, 2)`	`json.dumps(v, indent=2)`
Garder les accents	Par défaut	`ensure_ascii=False`
Trier les clés	À programmer	`sort_keys=True`
Erreur de lecture	`SyntaxError`	`json.JSONDecodeError`
Valider	Zod, AJV	Pydantic, `jsonschema`

Tableau 7 : Fonctions équivalentes en JavaScript et en Python.

Type	Usage
`application/json`	Cas général. Aucun paramètre `charset` n'est défini : l'UTF-8 est imposé.
`application/problem+json`	Erreurs d'API (RFC 9457)
`application/json-patch+json`	JSON Patch (RFC 6902)
`application/merge-patch+json`	JSON Merge Patch (RFC 7396)
`application/ld+json`	JSON-LD
`application/x-ndjson`	JSON Lines, très utilisé mais non enregistré à l'IANA (préfixe `x-`)

Tableau 8 : Types de média liés à JSON.

Références bibliographiques

[1] Tim Bray. The JavaScript Object Notation (JSON) Data Interchange Format. RFC 7159, March 2014. https://www.rfc-editor.org/rfc/rfc7159.

[2] Tim Bray. The I-JSON Message Format. RFC 7493, March 2015. https://www.rfc-editor.org/rfc/rfc7493.

[3] Tim Bray. The JavaScript Object Notation (JSON) Data Interchange Format. RFC 8259 (Internet Standard 90), December 2017. https://www.rfc-editor.org/rfc/rfc8259.

[4] Paul Bryan and Mark Nottingham. JavaScript Object Notation (JSON) Patch. RFC 6902, April 2013. https://www.rfc-editor.org/rfc/rfc6902.

[5] Paul Bryan, Kris Zyp, and Mark Nottingham. JavaScript Object Notation (JSON) Pointer. RFC 6901, April 2013. https://www.rfc-editor.org/rfc/rfc6901.

[6] Douglas Crockford. The application/json Media Type for JavaScript Object Notation (JSON). RFC 4627, July 2006. https://www.rfc-editor.org/rfc/rfc4627.

[7] Douglas Crockford. *JavaScript: The Good Parts*. O'Reilly Media, 2008.

[8] Ecma International. The JSON Data Interchange Syntax. Standard ECMA-404, 2nd Edition, December 2017. https://www.ecma-international.org/publications-and-standards/standards/ecma-404/.

[9] Ecma International. ECMAScript 2024 Language Specification. Standard ECMA-262, 15th Edition, June 2024. https://tc39.es/ecma262/.

[10] Stefan Gössner, Glyn Normington, and Carsten Bormann. JSONPath: Query Expressions for JSON. RFC 9535, February 2024. https://www.rfc-editor.org/rfc/rfc9535.

[11] Paul Hoffman and James Snell. JSON Merge Patch. RFC 7396, October 2014. https://www.rfc-editor.org/rfc/rfc7396.

[12] IEEE. IEEE Standard for Floating-Point Arithmetic. IEEE Std 754-2019, 2019. DOI: 10.1109/IEEESTD.2019.8766229.

[13] ISO/IEC JTC 1/SC 22. Information Technology – The JSON Data Interchange Syntax. ISO/IEC 21778:2017, 2017. https://www.iso.org/standard/71616.html.

[14] Michael B. Jones, John Bradley, and Nat Sakimura. JSON Web Token (JWT). RFC 7519, May 2015. https://www.rfc-editor.org/rfc/rfc7519.

[15] Gregg Kellogg, Pierre-Antoine Champin, and Dave Longley. JSON-LD 1.1: A JSON-based Serialization for Linked Data. W3C Recommendation, July 2020. https://www.w3.org/TR/json-ld11/.

[16] Martin Kleppmann. *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media, 2017.

[17] Mark Nottingham and Erik Wilde. Problem Details for HTTP APIs. RFC 7807, March 2016. https://www.rfc-editor.org/rfc/rfc7807.

[18] Mark Nottingham, Erik Wilde, and Sanjay Dalal. Problem Details for HTTP APIs. RFC 9457, July 2023. https://www.rfc-editor.org/rfc/rfc9457.

[19] Anders Rundgren, Bret Jordan, and Samuel Erdtman. JSON Canonicalization Scheme (JCS). RFC 8785, June 2020. https://www.rfc-editor.org/rfc/rfc8785.

[20] Yaron Sheffer, Dick Hardt, and Michael B. Jones. JSON Web Token Best Current Practices. RFC 8725 (BCP 225), February 2020. https://www.rfc-editor.org/rfc/rfc8725.

[21] Austin Wright, Henry Andrews, Ben Hutton, and Greg Dennis. JSON Schema: A Media Type for Describing JSON Documents (Draft 2020-12). Internet-Draft, 2020. https://json-schema.org/draft/2020-12/json-schema-core.html.